# Cloud Networking

## AWS Migrating a Web Application to Docker Containers

*Appunti spiegati passo passo*

### Fonte usata

PDF: “15 - CN - AWS Migrating a Web Application to Docker Containers”. Il documento presenta il Lab 8.1: migrazione di una web application e del backend MySQL verso container Docker, con caricamento finale delle immagini su Amazon ECR.

### Obiettivo degli appunti

Capire non solo cosa si fa nel laboratorio, ma perché lo si fa: si parte da un’applicazione distribuita su due istanze EC2 separate, si containerizzano applicazione Node.js e database MySQL, si testa la comunicazione tra container e infine si pubblicano le immagini su Amazon Elastic Container Registry.

# 1. Idea generale del laboratorio

**Il laboratorio mostra una migrazione applicativa verso Docker.** All'inizio l'applicazione di inventario del fornitore di caffè è eseguita direttamente su istanze EC2: una istanza ospita l'applicazione Node.js e un'altra istanza ospita MySQL. Alla fine, entrambi i componenti vengono trasformati in container Docker eseguiti su un host con Docker Engine.

| Aspetto | Scenario iniziale | Scenario finale |
|---|---|---|
| Unità di deployment | Applicazione installata direttamente nel sistema operativo guest di EC2. | Applicazione e database pacchettizzati come immagini Docker e avviati come container. |
| Dipendenza dall'ambiente | Alta: librerie, versioni e configurazioni sono legate all'istanza. | Più bassa: il container include dipendenze e runtime necessari. |
| Portabilità | Limitata: spostare l'app richiede ricreare l'ambiente. | Maggiore: l'immagine Docker può essere eseguita su host compatibili. |
| Distribuzione | Manuale o legata alla configurazione delle istanze. | Basata su build, tag, run e push delle immagini. |
| Repository immagini | Non presente. | Amazon ECR come registry gestito per conservare e distribuire immagini Docker. |

**Da ricordare**

Docker non elimina la necessità di un host: i container devono comunque girare sopra un sistema operativo con Docker Engine. La differenza è che l'applicazione non viene più installata "a mano" sull'istanza, ma viene incapsulata in un'immagine riproducibile.

# 2. Scenario iniziale

Nel diagramma iniziale del lab sono presenti tre blocchi principali:

- AppServerNode EC2 instance: istanza EC2 che esegue la node app sopra il sistema operativo guest.
- MysqlServerNode EC2 instance: istanza EC2 separata che esegue MySQL sopra il sistema operativo guest.
- IDE running on EC2 instance: ambiente usato come container host, già dotato di Docker Engine.

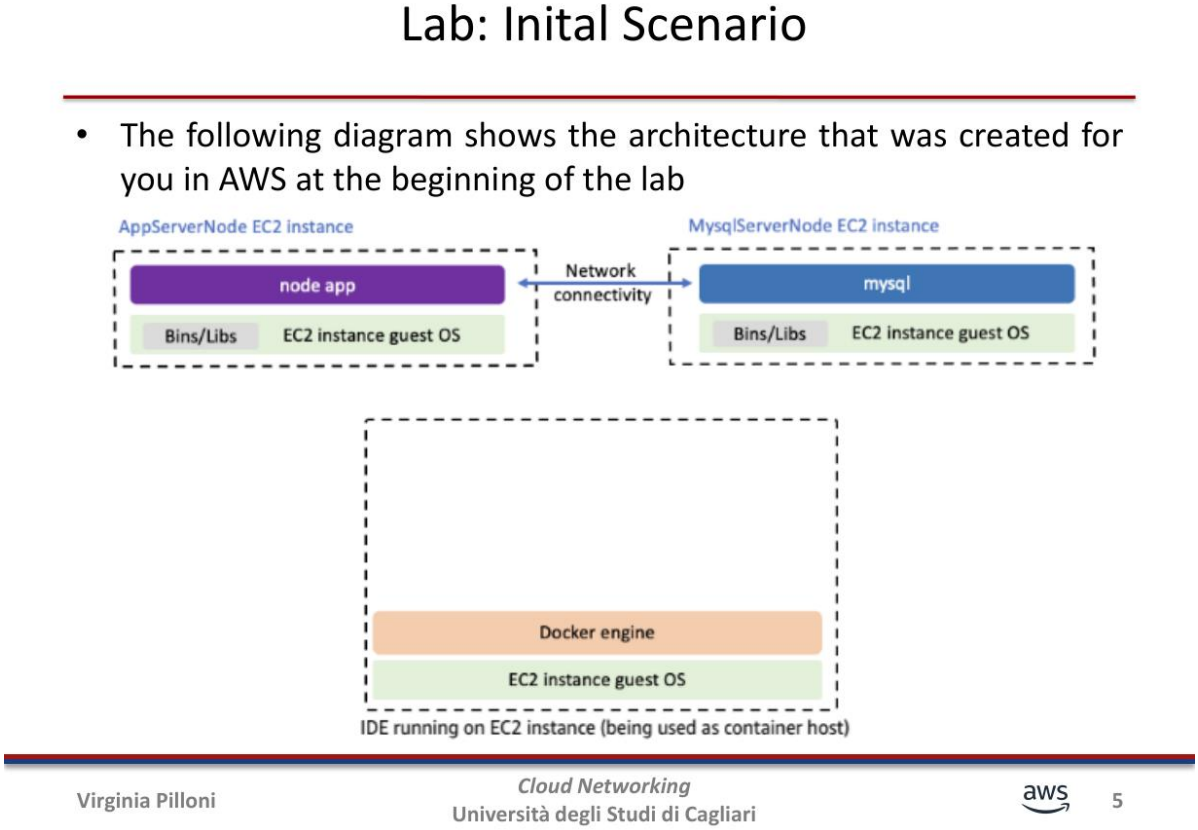



*Figura 1 - Scenario iniziale del laboratorio: applicazione Node.js e MySQL sono ancora installati su istanze EC2 separate; l'istanza IDE contiene Docker Engine e sarà usata come host dei container.*

**Analisi del flusso iniziale**

Client o tester → node app su AppServerNode → connessione di rete verso MysqlServerNode → database MySQL. La logica applicativa e il database sono distribuiti su due macchine EC2 distinte, quindi configurazione di rete, porte, sicurezza e dipendenze software devono essere coerenti tra host diversi.

| Problema potenziale | Perché è rilevante | Effetto pratico |
|---|---|---|
| Dipendenze installate sull'host | L'applicazione dipende dal sistema operativo guest, dalle librerie e dalle versioni installate. | Migrazione e riproduzione dell'ambiente più lente. |
| Database su host separato | Richiede connettività di rete corretta e apertura delle porte necessarie. | Errori di connessione se security group, hostname o porta non sono configurati bene. |
| Scalabilità non automatica | Aggiungere nuove istanze richiede ulteriori configurazioni. | Risposta lenta a picchi di carico. |
| Deployment poco standardizzato | Non esiste un artefatto unico e versionabile dell'applicazione. | Difficile ricreare lo stesso ambiente in modo identico. |

## 3. Concetti necessari prima della migrazione

| Concetto | Spiegazione semplice | Ruolo nel laboratorio |
|---|---|---|
| Container | Ambiente isolato che contiene applicazione e dipendenze necessarie. | Serve per eseguire node app e MySQL senza installarli direttamente sull'host EC2. |

| | | |
|---|---|---|
| Immagine Docker | Template immutabile da cui vengono creati i container. | Viene costruita per l'applicazione Node.js e per il database MySQL. |
| Dockerfile | File di istruzioni per costruire un'immagine Docker. | Descrive base image, dipendenze, copia del codice, porte e comando di avvio. |
| Docker Engine | Software che crea, avvia e gestisce container. | È installato sull'istanza IDE, che diventa container host. |
| Port mapping | Associazione tra porta dell'host e porta interna del container. | Permette di raggiungere l'applicazione dall'esterno, ad esempio host:8000 → container:3000. |
| Variabili d'ambiente | Parametri passati al container al momento dell'avvio. | Utili per indicare all'app il nome host del database, password, utente o database. |
| Amazon ECR | Registry gestito AWS per immagini container. | Riceve le immagini finali dopo build, tag e push. |

**Differenza chiave: VM vs container**

Una VM virtualizza un intero sistema operativo guest sopra un hypervisor. Un container condivide il kernel dell'host e include solo il necessario per eseguire il processo applicativo. Per questo i container sono più leggeri e più rapidi da avviare, ma richiedono comunque un container engine e una corretta configurazione di rete/storage.

# 4. Svolgimento passo passo del Lab 8.1

## Task 1 - Preparare l'ambiente di sviluppo

Si verifica che l'ambiente EC2 usato come IDE abbia gli strumenti necessari: terminale, repository di codice, Docker Engine, AWS CLI e permessi IAM. Questo passaggio è preliminare: senza Docker non si possono costruire immagini né eseguire container, senza AWS CLI/ECR non si possono pubblicare le immagini.

## Task 2 - Analizzare l'infrastruttura esistente

Si osserva come è fatta l'applicazione prima della migrazione: app Node.js su una EC2, MySQL su un'altra EC2, con connettività di rete tra le due. Lo scopo è capire quali componenti devono essere containerizzati e quali parametri devono rimanere coerenti: porte, hostname, credenziali, nome del database e dipendenze.

## Task 3 - Migrare l'applicazione a un container Docker

Si crea un'immagine per la node app. Tipicamente il Dockerfile parte da una base image Node.js, copia il codice dell'applicazione, installa le dipendenze e definisce il comando di avvio. Poi si costruisce l'immagine con docker build e si avvia un container con docker run, esponendo la porta applicativa.

## Task 4 - Migrare il database MySQL a un container Docker

Si crea o usa un'immagine MySQL configurata con database, utente, password e dati iniziali. Il container MySQL deve esporre la porta 3306 e, in un caso reale, dovrebbe usare un volume persistente per non perdere i dati quando il container viene rimosso.

## Task 5 - Testare il container MySQL con l'applicazione Node.js

Si avvia la node app indicando come host del database il container MySQL o l'indirizzo raggiungibile nel network Docker. Il test serve a verificare che l'applicazione riesca a interrogare il database, che le credenziali siano corrette e che le porte siano aperte.

## Task 6 - Aggiungere le immagini Docker ad Amazon ECR

Le immagini locali vengono taggate con l'URI del repository ECR e poi caricate con docker push. In questo modo diventano artefatti riutilizzabili in ambienti successivi, ad esempio ECS, EKS, Elastic Beanstalk o pipeline CI/CD.

| Fase | Comando/logica tipica | Scopo |
|---|---|---|
| Build app | docker build -t node-app . | Creare un'immagine della web application. |
| Run app | docker run -p host:container ... node-app | Avviare il container e pubblicare la porta. |
| Run DB | docker run -e MYSQL_* -p 3306:3306 mysql-image | Avviare MySQL in container con configurazione iniziale. |
| Test | curl / browser / query app | Verificare che app e database comunichino. |
| Tag ECR | docker tag local-image account.dkr.ecr.region.amazonaws.com/repo:tag | Preparare l'immagine per il registry AWS. |
| Push ECR | docker push account.dkr.ecr.region.amazonaws.com/repo:tag | Pubblicare l'immagine nel registry gestito. |

## 5. Scenario finale

Alla fine del laboratorio i due vecchi nodi applicativi non sono più il punto in cui viene eseguita direttamente l'applicazione. L'app Node.js e MySQL sono stati migrati in due container distinti sopra Docker Engine, all'interno dell'istanza EC2 usata come container host.

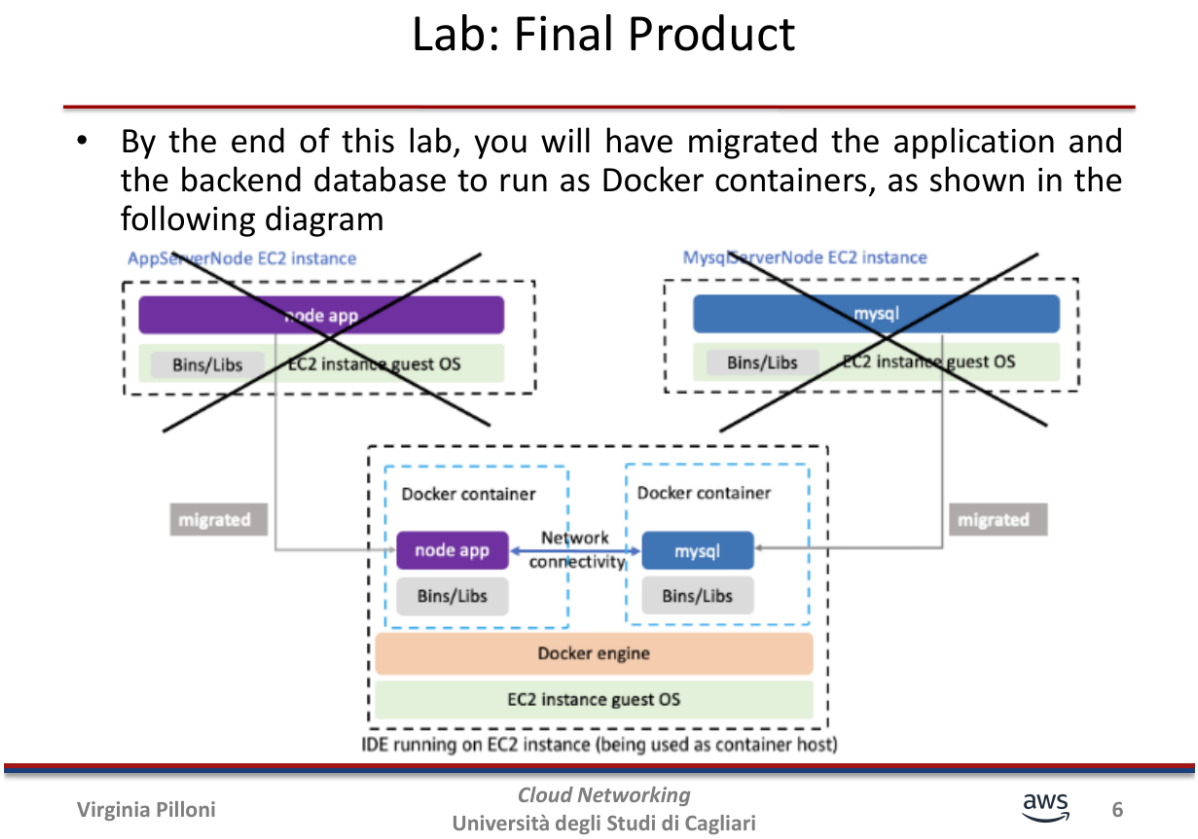



*Figura 2 - Prodotto finale del laboratorio: la node app e MySQL sono ora container Docker. I vecchi ambienti EC2 originari risultano superati dalla nuova modalità di deployment containerizzata.*

**Flusso finale dei dati**

Richiesta utente/tester → porta pubblicata sull'host Docker → container node app → rete Docker interna → container MySQL → risposta del database → node app → risposta al client.

**Schema logico finale**

```
[Client/Test]
    |
    v
[EC2 container host / IDE]
    |
    +--> [Docker container: node app]
             |
             v
        [Docker network]
             |
             v
        [Docker container: MySQL]

[Immagini Docker locali] --> tag --> push --> [Amazon ECR]
```

## 6. Analisi architetturale

| Componenti | Ruolo | Criticità da controllare |
|---|---|---|
| EC2 container host | Fornisce sistema operativo e Docker Engine per eseguire i container. | Se è un singolo host, diventa single point of failure. |
| Container node app | Esegue la logica applicativa web/inventory. | Deve ricevere correttamente host DB, porta e credenziali. |
| Container MySQL | Esegue il backend database. | Richiede persistenza dati, backup e protezione delle credenziali. |
| Docker network | Consente la comunicazione tra container. | Nome host, bridge network e porte devono essere configurati correttamente. |
| Port mapping | Espone il servizio applicativo all'esterno dell'host. | Aprire solo le porte necessarie e proteggere con security group. |
| Amazon ECR | Conserva le immagini container versionate. | Richiede autenticazione, policy IAM e lifecycle management. |

| Bottleneck / rischio | Analisi | Raccomandazione |
|---|---|---|
| Singolo host Docker | Tutti i container girano sullo stesso host. Se l'host fallisce, app e DB non sono disponibili. | Per produzione usare ECS/EKS/Fargate con più task/pod su più Availability Zone. |
| Database containerizzato senza volume | Se il container viene eliminato, i dati possono andare persi. | Usare volumi persistenti oppure, meglio, RDS/Aurora per database gestito. |
| Credenziali in variabili d'ambiente | Comodo per il lab, ma non ideale per segreti reali. | Usare AWS Secrets Manager o Parameter Store. |
| Sicurezza porte | Esporre porta app o MySQL in modo troppo ampio aumenta la superficie d'attacco. | Aprire solo le porte richieste; MySQL non dovrebbe essere pubblico. |
| Immagini non versionate | Tag generici come latest rendono difficile il rollback. | Usare tag semantici o hash di commit. |
| Assenza di monitoraggio | Il lab verifica la funzionalità, ma non introduce monitoraggio completo. | Aggiungere CloudWatch logs/metrics e allarmi. |

## 7. Come trasformare il lab in una soluzione più realistica

| Esigenza | Soluzione AWS consigliata | Motivazione |
|---|---|---|
| Eseguire container senza | AWS Fargate | Riduce overhead amministrativo: non si |

| | | |
|---|---|---|
| gestire server | | gestiscono direttamente le istanze EC2. |
| Orchestrare più container | Amazon ECS o Amazon EKS | Gestiscono deployment, scaling, restart, service discovery e integrazione con load balancing. |
| Conservare immagini | Amazon ECR | Registry privato integrato con IAM e servizi container AWS. |
| Database affidabile | Amazon RDS/Aurora | Backup, replica, patching e disponibilità gestiti dal provider. |
| Bilanciare traffico | Elastic Load Balancing | Distribuisce le richieste su più task/container/istanze. |
| Osservabilità | Amazon CloudWatch | Metriche, log e allarmi per individuare guasti o saturazioni. |
| Gestione segreti | AWS Secrets Manager / Parameter Store | Evita password hard-coded o esposte nei comandi di avvio. |

**Sintesi critica**

Il laboratorio è ottimo per capire la meccanica della containerizzazione, ma non rappresenta da solo una soluzione production-grade. La migrazione a Docker migliora portabilità e riproducibilità; per alta disponibilità, scalabilità e sicurezza servono orchestrazione, database gestito, bilanciamento, monitoraggio e gestione sicura dei segreti.

## 8. Cose da sapere per l'esame

- Saper descrivere la differenza tra applicazione installata direttamente su EC2 e applicazione containerizzata.
- Saper spiegare il ruolo di Docker Engine, immagine, container, Dockerfile e registry.
- Saper disegnare scenario iniziale e finale della migrazione.
- Saper motivare perché Amazon ECR è utile nella fase finale.
- Saper individuare le criticità: persistenza del database, sicurezza delle porte, single point of failure, gestione segreti.
- Saper proporre un'evoluzione production-grade con ECS/EKS/Fargate, RDS, ELB e CloudWatch.

## 9. Domande d'esame con risposta

1. **Qual è l'obiettivo principale del Lab 8.1?**

   **Risposta:** Migrare una web application e il suo database MySQL da un'esecuzione diretta su istanze EC2 a un'esecuzione tramite container Docker, con pubblicazione finale delle immagini su Amazon ECR.

2. **Nello scenario iniziale quali componenti sono presenti?**

   **Risposta:** Una istanza EC2 AppServerNode con la node app, una istanza EC2 MysqlServerNode con MySQL e una istanza EC2 usata come IDE/container host con Docker Engine.

3. **Perché la containerizzazione rende il deployment più portabile?**

   **Risposta:** Perché l'immagine Docker contiene applicazione, dipendenze e istruzioni di avvio. Lo stesso artefatto può essere eseguito su host compatibili senza ricostruire manualmente l'ambiente.

4. **Che cosa rappresenta una Docker image?**

   **Risposta:** È un template immutabile da cui vengono creati uno o più container. Nel lab vengono create immagini per l'applicazione e per il database.

5. **Che cosa rappresenta un container Docker?**

**Risposta:** È un'istanza eseguibile di un'immagine. Nel lab i container finali eseguono rispettivamente la node app e MySQL.

6. **Qual è il ruolo di Docker Engine?**

   **Risposta:** È il motore che crea, avvia, ferma e gestisce i container sull'host. Nel lab è installato sull'istanza EC2 usata come container host.

7. **Perché bisogna testare la comunicazione tra node app e MySQL?**

   **Risposta:** Per verificare che rete Docker, porte, hostname, credenziali e nome del database siano configurati correttamente. Se uno di questi elementi è errato, l'applicazione non riesce a interrogare il database.

8. **A cosa serve Amazon ECR in questo laboratorio?**

   **Risposta:** Serve come container registry gestito da AWS: conserva le immagini Docker caricate dal laboratorio e permette di riutilizzarle in servizi successivi come ECS, EKS, Fargate o pipeline CI/CD.

9. **Qual è il principale limite dello scenario finale del lab?**

   **Risposta:** App e database girano sullo stesso container host. Questo è utile per imparare, ma in produzione introduce un single point of failure e non garantisce alta disponibilità.

10. **Perché un database in container richiede attenzione particolare?**

    **Risposta:** Perché i dati devono sopravvivere al ciclo di vita del container. Senza volumi persistenti, backup o servizio database gestito, la rimozione del container può causare perdita dati.

11. **Quale servizio AWS useresti per rendere più production-grade il database?**

    **Risposta:** Amazon RDS o Aurora, perché offrono gestione automatizzata di backup, patching, replica, disponibilità e recovery rispetto a un database MySQL eseguito manualmente in container.

12. **Quale servizio AWS useresti per orchestrare i container?**

    **Risposta:** Amazon ECS o Amazon EKS. ECS è più integrato e semplice in AWS; EKS è adatto quando si vuole Kubernetes. Con Fargate si può eseguire il workload containerizzato senza gestire server.

13. **Qual è la differenza tra build, run, tag e push?**

    **Risposta:** Build crea l'immagine, run avvia un container da quell'immagine, tag assegna un nome compatibile con il repository di destinazione, push carica l'immagine nel registry remoto come Amazon ECR.

14. **Quali controlli di sicurezza sono importanti dopo la migrazione?**

    **Risposta:** Limitare le porte aperte con security group, non esporre MySQL su Internet, usare IAM per ECR, gestire segreti con servizi dedicati, aggiornare le immagini e controllare vulnerabilità.

## 10. Schema finale da memorizzare

**Schema da esame**

Prima:
EC2 AppServerNode -> node app installata nel guest OS
EC2 MysqlServerNode -> MySQL installato nel guest OS
Connessione di rete tra le due istanze

Dopo:
EC2 container host -> Docker Engine
Docker container 1 -> node app
Docker container 2 -> MySQL
Docker network -> comunicazione interna tra container
Amazon ECR -> immagini Docker pubblicate e riutilizzabili